

Course Overview

Course Overview

Hi, everyone! I'm Mike Woodring, and this is C#: The Big Picture. C# has been one of the top 10 most popular programming languages in the world for many years now, so demand for people with C# fluency and skills is very high, and for good reason. As a general purpose cross-platform language that supports both object-oriented and functional programming paradigms, it can be used to build apps that run on desktops, mobile devices, and the web, as well as games, and even PowerShell cmdlets. This course is a technical introduction to the C# programming language. But while this course is technical in nature, it is more about the context of C# rather than the line-by-line details of using C# to build a particular type of app. We'll leave that for other courses in the library. Although you'll see plenty of C# along the way, and even a bit of a similar language if you choose, you don't have to worry about absorbing every little detail. You can sit back, relax, and hopefully just enjoy getting the gist of what programming in C# looks like. This course is primarily aimed at developers and other technologists who are at the earliest stages of C# evaluation and adoption. This includes people who are just evaluating C#'s fit for their needs, or who have already made that decision and are about to embark on a learning journey towards C# proficiency. It's also for people who will simply be working with C# developers and who just need to acquire some conversational familiarity with the core concepts of the language and its relationship with the .NET platform. By the end of this course, you'll have discovered C#'s syntactic style, what's required in order for C# code to execute, how safe in performance C# code is when run, and what to expect going forward as C# continues to evolve. In other words, you'll have the big picture. I hope you'll join me on this journey of discovery about C# with this course, C#: The Big Picture at Pluralsight.

Discovering C#

Introduction

I said this course is about context, so I want to start this exploration of the C# language by framing it within the context of assumptions I'm making about you, my perspective on things, so that you're aware of my biases, and a little bit more about the approach I'm taking in this course.

A Little Bit About You

To start, I'm going to assume that you are a technologist at least somewhat familiar with programming. Maybe you're a developer somewhere on the spectrum from newbie to veteran. Or maybe you're not a programmer at all, but you do have at least a conversational understanding of basic programming concepts, like branching, loops, and functions, and at least a basic idea of what classes are about. Since you're watching this course, I assume you're curious to know more about C#, but not necessarily because you need to become a C# practitioner. I'm also going to assume you're skeptical, in the most complimentary sense. I think part of what makes developers good developers is a healthy resistance to taking bold statements about a programming language's benefits at face value. So I will avoid making bold statements about C#'s benefits without illustrating the point or showing you how you can convince yourself. Lastly, I assume you either aspire to be a C# developer or perhaps just someone who can communicate more effectively with C# developers. And so at this point, you just want to understand the essence of what makes C# tick as opposed to spending a lot of time in the details.

A Little Bit About Me

I believe turnabout is fair play, so let me paint myself with the same brush I just painted you with, mostly so that you know what my biases and perspective are, which is only fair. I've been a developer for a long while now, so I've encountered a lot of programming language features and paradigms over the years, which colors, how have you any new hotness? But after 30 years of learning and using new languages, I'm still curious. I love to learn about new programming languages or new language features. So although I began my relationship with C# as an avowed skeptic, I've since become a real fan. So you should know that my goal in this course is not to provide a dispassionate impartial introduction to C# what I'm going to do is highlight some of the features that make me a fan, some of which may convince you to become a fan too.

A Little Bit (More) About This Course

With all of that in mind, this course is less about the syntax of C# and more about the reasons why bothering to learn it's a syntax might be useful to you. So my goal with this course is to kick-start your discovery of CRP by taking the following approach in this course. As I said earlier, there is a lot of surface area to C#, so I'm going to stick to the ABCs and keep things at a very elemental level. I'll leave the applied C# details to other courses in the library, which I'll reference throughout the course. This means I'm going to take some liberties with what I show onscreen in order to keep things moving and stay at the big picture level. My goal is to convey the flavor of C#, not provide screenfulls of code that you can execute on your own machine, although all of my code is available within the exercise files that accompany this course for those of you with access. And because I'm a skeptic who believes you deserve to have big claims about C# backed up by evidence, I'm going to pull back the covers somewhat and reveal just how some of C#'s most powerful features are implemented. And for those of you who are aspiring C# developers, I hope to help you come away with an appreciation for how C# has handled the inevitable evolution of a language over time, and an understanding of what to expect as it continues to evolve moving forward.

Tools Used in This Course

For those of you who do want to experiment with anything you see in this course, these are the tools that were used at the time of this recording. I'm using the latest version of the C# compiler that is currently in general release. There may be a preview of the next version of C# available, but I'm not using that for this course. As we'll explore in the next module, this is the underlying runtime and accompanying set of Base Class Libraries that complement the compiler I'm using. And as for the integrated development environment, I'm using the free version of Visual Studio that anyone watching this course can download and install. If you download and install this version of Visual Studio, you will get all three of these things installed and ready to use out of the box and will be able to replicate everything you see in this course. Importantly, although this course was filmed using the latest long-term release of Microsoft's compiler runtime and integrated development environment, all of the key runtime and language concepts covered in this course are applicable to every version of C# released over the last 20+ years. In terms of the specific syntax shown throughout the course, the majority is also applicable to all versions of C#, with the obvious exception of any version-specific

syntax I used to illustrate how C# has evolved over the years. For those few examples, I'll clearly state which version of the compiler introduced a particular feature. So if you need to use an older version of the compiler for some reason, you'll need to use the older version of the syntax for that feature. And although I use Microsoft's tool set to demonstrate a few things along the way, the open source nature of this technology means that there are a vast number of alternative tools available, a few of which are listed here.

Historical Context

We'll start by taking a look at the essence of C#, those general features and characteristics that summarize its nature as a programming language and give you a sense of the experience you'll have programming with it. As is so often the case, historical context has a lot to do with why something is the way it is now. C# is no different in this regard that said using it to build apps that ran on more than one platform wasn't the easiest to pull off and its syntax seemed to be increasingly complex as the language evolved. Java, on the other hand, had acquired quite a large following in a relatively shorter period of time. And although generating fast code wasn't one of its strengths back then, it was appreciated for being much more portable across platforms and arguably more productive for building complex yet still maintainable systems. As an aside, one aspect of Crp's contextual origin story is how it came to be called C#, which was a bit of an odd choice at the time, which could be simplified like so which looked remarkably similar to the sharp symbol on a keyboard, go C#

Approachable

This meant that C# would be in the C family of languages, which meant things like using semicolons rather than new lines as statement terminators using curly braces to group blocks of code comprised of more than one statement that should be considered to be in the same scope supporting object oriented constructs such as namespaces, classes, methods and properties. And additionally using zero based indexing when working with array.

Strongly Typed

So in C#, the type of things like method return values, method arguments and local variables must be declared in code. While fans of loosely typed languages find strongly typed languages somewhat cumbersome to work with the effort pays off with the prevention of errors at both compile time and runtime, which is a significant benefit. And one will revisit shortly that said the C# compiler is very often able to infer the type of a variable or parameter by analyzing the context of its usage. In this case, the type of variable N can be unambiguously inferred to be the same as the type of the literal zero that is used to initialize it, which in this case is an N in C# developers signal that they want the compiler to infer the type of something by using the `var` keyword instead of specifying a concrete type. So long as there is nothing ambiguous about a type inference expression, the C# compiler will substitute the concrete type. And in this case, when it compiles your code, if we open this code in a Visual Studio, we can verify that the compiler is correctly inferring the type of N as expected a couple of ways. First, if we hover over the loop variable N that was declared using the `var` keyword. Visual Studio displays a tool tip that indicates its type isn't. And if we hover over the literal value zero, that is being used to initialize that loop variable Visual Studio informs us that its type is `System.Int32` which is the .NET representation of a 32 bit signed integer. That and the fact that N is being compared to the length property of the ARG parameter, which is also an integer gives the C# compiler enough information to unambiguously infer that the type of local variable N should be an integer. Secondly, we can confirm that type information is made available at runtime by inspecting things in the debugger to do that. I'll run to the point where the for loop begins. Notice that the locals window also indicates that the type of N is an `int` and we can go one step further using the debuggers intermediate window to programmatically fetch the name of N's underlying type. Now, in this program, the coding convenience we just achieved is literally a wash since `int` and `var` both involve typing exactly three letters but type inference pays much bigger dividends when working with custom types, especially when working with something called generics, which is a subject for another course in the library. You'll also see more examples of type inference in just a moment. And throughout the rest of this course the fact that this kind of type information is available at runtime, not just compile time is what makes the next key feature of C# possible.

Resilient and Safe

The fact that C# is statically and strongly typed sets the stage for what I would argue as one of the biggest features of C#, which is that it exhibits all of the resilience to change in runtime safety and that traditionally comes with more loosely typed dynamic languages, but retains the performance characteristics traditionally exhibited only by statically typed natively compiled code. Now, that's one of those bold claims which should cause your skeptic antenna to twitch, but the proof is somewhat involved, so I'm going to leverage a bit of foreshadowing and park the native performance statement on the back burner for now and focus here on the safety and resilience characteristics. But only just for now. I promise that I will come back in the next module and show you exactly how C# is able to achieve the performance characteristics of native code. Consider this slight variation of our rudimentary C# program. All this program does is declare an array of integers populated with the numbers 1 through 5, declare a variable named sum that is initialized to the integer value of 0, and then loop over that array of numbers, adding the contents of each element of the array to our sum variable. Notice that we're using the Length property of the numbers variable to determine how many times our for loops should iterate. This is one of the productivity and safety features that most strongly typed natively compiled languages don't provide. Once the loop concludes, the tally we've accumulated in the sum variable is displayed to the console. Let's switch over to Visual Studio and use this to explore one example of what I mean by safe. Okay, I've switched to Visual Studio and have a simple C# project ready that reproduces the code shown on the previous screen. As expected, if I build and run this program, the output is 15. Likewise, if I change the loop such that it only iterates four times as a hard-coded value, the output is 10, as expected. But what happens if I now change how I initialize the numbers variable so that it only has 3 values in it, like so? But if you were coming from a Java background, you would probably guess or at least hope that this kind of error results in an exception of some sort indicating that your program has attempted to access the given array at an out of bounds location. Thankfully, C# works like Java in this regard and an index out of range exception occurs. This is actually a good thing in the case of this simple program, this kind of runtime type safety prevents incorrect output. But if you're unlucky which is more typical that corruption would go undetected for potentially quite a long time resulting in undefined program behavior that can be exceedingly difficult to pin down. That's one example of the safety of C#. Now, let's look at what I mean by resilience to demonstrate C Sharp's resilience. I've set up a new application that uses a very simple two dimensional point structure that's been defined in a separate library plural site has other

courses that go into more detail about how libraries are created and used in C# for now. The important bit is that the definition and implementation of the point structure is in a separate binary which allows it to be reused across multiple applications including this one. All the application does is initialize a local variable called PT set its X and Y fields to initial values and then display the result of calling the point structures two string method to the console. Notice that when this version of the program is run, the implementation of two string returns a string that shows the xy coordinates of the point in standard parentheses notation. Now let's use this simple setup to replicate a very common occurrence in software systems which is at a library we are using is updated to a newer version but deployed to systems that still have the original version of the calling application in use in this demo deployment just means copying a version of the point library into the folder. Where the main application executable resides. We'll replicate that scenario by changing our two dimensional point into a three dimensional point by adding a new field that will hold the Z coordinate for the point we're going to rebuild just the library and then quote unquote, deploy the updated library to the location where our app was installed without rebuilding the application. If we rerun our app, which we have not changed or recompile to be aware of the new field, the application still runs as expected albeit with a Z field of zero. This demonstrates C# code's resilience to change when the application developer is ready to opt in to taking advantage of the new functionality available in the library. They are using, they can of course rebuild and redeploy their application and now have access to the extended functionality. This ability to maintain resilience in the presence of non breaking changes to dependencies is a hallmark of C# and other managed languages. And to revisit the previous point about safety, should a library developer make some sort of breaking change to a type definition which i'll demonstrate here by changing the type of the fields in our point structure from integers to strings. An exception would result since the expectations encoded in the dependent application about the nature of the point structure can no longer be guaranteed by the runtime. If you're a skeptic or merely observant, you recall that I made a fairly bold claim about CRP being resilient and safe with native performance. We'll explore exactly how safety and resilience are pulled off and revisit the native performance boost in the next module on managed execution.

Object-oriented with Functional Features

For now, let's turn the corner on our initial discovery tour and touch on C#'s support for multi-paradigm programming. Consider again our simplistic array of numbers program. To demonstrate C#'s inherent object-orientedness, I'm going to forego defining some sort of base class and derived class hierarchy and again leave that to other courses in the library. Instead, I'll simply point out that at some point, every type in C# derives from or extends a standard .NET type named `System.Object`, which, among other things, provides a method named `GetType` that you can use at runtime to ask any object for information about its type. This is done by calling the `GetType` method, which we leveraged in an earlier demo. Having done that, we can display the name of the type to the console by accessing its `FullName` property, which, in this case, turns out to be a type called `System.Int32` with array brackets after it. This denotes it's an array of 32-bit signed integers. This is an example of a C# language keyword or construct being mapped to a standard type in the .NET Base Class Library, which is another way that C# endeavors to be approachable to programmers familiar with other C-based programming languages. There are numerous such language mappings in C#. We can also go one step further. As it turns out, not only can we use our type reference to determine the name of this type, we can also use the `BaseType` property of any objects type to determine whether that type derives from or extends some other type. And if we do this in a loop like so, we can walk the entire inheritance tree until we reach the root of our numbers array type hierarchy, which, when run, would produce this output. I'll include references to additional courses on C#'s type system at the end of this module. For now, let's examine what I mean by with functional features. Let's return to our original version of our array of numbers program, which computed and then displayed the sum of the integers within the array. Our original implementation used a standard `for` loop to iterate over each element of the array, adding the current array elements value to the running tally being stored in the `sum` variable. While this is a perfectly functional way to implement this algorithm, pun intended, C# allows developers to embrace, if they choose, a more functional programming paradigm. In functional programming languages, executable expressions or functions are first class entities within the syntax of the language and can be treated like any other typed object, including being referenced by variables and passed into and out of other functions as parameters and return values. In C#, the functional equivalent of the previous implementation would look like this. Instead of writing a traditional `for` loop, we would instead simply invoke an `Aggregate` method on our array of numbers. The `Aggregate` method takes two parameters as input. The first parameter is simply the initial value of our

accumulator, which, in our case, is just 0, and the second parameter is an expression of the function we would like the Aggregate method to invoke for each element in the numbers array. If you look at the documentation for the Aggregate method, you'll find that there are three elements to this expression. A parameter list, followed by a fat arrow operator, and then the set of statements we would like to execute for each element in the input array. Together this line of code forms what C# refers to as an expression lambda. The documentation for the Aggregate method indicates that it will call our functional expression with two input arguments, the first being the current value of the accumulator, and the second being the value of the current element in the input array. While the order of those parameters is defined by the aggregate function, we can name those parameters anything we like. In this case, total and num are apt names for those two arguments. The body of our anonymous method then simply returns the result of adding the current value to the running total. Since our anonymous method consists of just a single statement, we don't need to actually use the return keyword. It's implied in this case. And since this syntax is really just shorthand for defining an anonymous method that contains the required parameter list and a method body that contains whatever it is you want to execute, should you need to do something involving multiple statements, you can use a standard pair of curly braces to denote the collection of statements you would like to comprise that anonymous method. Note that when using this variation, which is officially referred to as a statement lambda, the return keyword is required. This is because the code within the curly braces can be arbitrarily complex, including the use of multiple return statements, so the compiler will no longer assume which statement should result in the return value for the functional expression. Now, I'm glossing over quite a bit of detail here, but the Aggregate method shown here is actually defined in another namespace called System.Linq, where LINQ is actually an acronym that stands for Language INtegrated Query. LINQ is beyond the scope of this course, but I'll include a reference at the end of this module to a course on LINQ that you can check out to learn more.

Open-source, Cross-platform, and General Purpose

What we've looked at so far is just the very topmost tip of the proverbial iceberg. Although I'm sticking to the ABCs of the language here and just using rudimentary console applications to demonstrate things, C# is much, much larger and more capable than what you see here. Since 2014, C# has been available as an open source project and supports the cross-platform development of console apps and

web apps that target Windows, macOS, and Linux. C# is also quite general purpose and can be used to build everything from desktop apps, to mobile apps for Android and iOS, to web apps, and even games and automation plugins for Microsoft's PowerShell technology, all of which are covered by other courses in the library.

Conclusion

And the combination of its strong typing and runtime type safety means that C# is very resilient to change and yet safe when coding errors or breaking changes are introduced. And although C# has its roots as an object oriented language and fully supports classic domain driven design, it has evolved to support more functional approaches to programming as well. All of which is being done as an open source cross platform project and which can be used to meet a variety of application development needs throughout this module I either mentioned or alluded to these courses in the plural side library that you can check out for more information on several of the topics I just touched on. I promised earlier that I would not forget to return to the bold claim about CRP being resilient and safe. But with native performance, making good on that promise is exactly what I'll do in the next module.

Exploring Managed Execution in C#

Introduction

Before I make good on my promise to explain the native performance claim I made earlier, I need to zoom out a bit and explain what the managed execution portion of this module title is referring to, which entails a tour of how the source code you type into an editor or IDE gets transformed into something a computer or device CPU can actually execute.

Code Execution Models

And notably compiled languages require the developer to manually manage memory usage. Those source files are then fed into a program called a compiler which converts the source code text into CPU specific binary code that is appropriate for a given platform on which the developer intends to run their program. Because the contents of such executable files are native to a given processor architecture. The operating system on the target machine can essentially feed the contents of a native executable image directly to the CPU which the CPU then executes if the application has a function that is called 1000 times. For example, this CPU simply executes the binary encoding of that method 1000 times. Absolutely no time is spent preparing that code to execute as the source code to machine code translation was handled by the compiler long before that application was deployed and run. This approach yields excellent runtime performance that said there are several downsides to this type of architecture including a lack of resilience to change that we looked at in the previous module. The lack of portability and the need to build one's code using multiple compilers in order to generate versions of the app that can be run on different platforms and the requirement that the programmer manually manage the allocation and the allocation of memory and other resources used throughout the life of the program, which is to put it mildly an error prone endeavor. Now, in keeping with the theme of this course, I'm simplifying the process quite a bit here. But hopefully that gives you a taste for how code comes to be executed when using a compiled language. In contrast, interpreted or dynamic languages lie at the very opposite end of the code execution model spectrum interpreted languages tend to be very loosely typed. Ranging from a complete absence of type to languages where an entity's type is determined at runtime and can then change over time as the program runs. While the burden of memory management is alleviated for the programmer and things like portability and productivity tend to be great performance is usually the most noticeable trade off with interpreted languages with hard to find bugs due to type permissiveness being a close second. This is because although the development process starts the same way using an editor or IDE to create text files containing something like Python source code, everything else involved in translating high level source code into processor specific instructions happens within an interpreter which is a program that runs on the target machine. And generally speaking, converts high level language statements into machine code on a line by line basis So if the application has a method that is called 1000 times the interpreter's translation of source code to machine code for that method may be happening 1000 times. Again, I'm oversimplifying here. There are interpreters that are smarter than that and seek to detect when they

can translate source code to machine code more efficiently. But the design of the language itself in terms of features and syntax tends to limit the efficiency gains that can be achieved. However, it is this type of architecture that allows interactive coding experiences like Python's `repl` or `read eval print` loop and portable jupyter notebooks for data centric processing goodness, which brings us to a somewhat hybrid system that I'm referring to in the module title as managed languages, managed languages such as C# tend to be strongly typed, similar to their compiled language cousins. But unlike their compiled cousins, managed languages, make all of the type of information that is available to developers while they are coding also available at runtime. Such runtime availability of type information allows for far more robust safety checks and the convenience of automatic memory management via something like a garbage collector, which is more akin to an interpreted language experience. While also making it possible to achieve much faster performance profiles with managed languages, developers run their code through a compiler similar to the compiled language model. However, managed compilers do not produce machine code that can be processed directly by device CPUS instead managed compilers produce a binary file that contains the intermediate representation of the high level source code, which in the case of C# is called intermediate language or just IL Java compilers work similarly, except that their output is referred to as bytecode. In CS Sharp's case, the file produced by the compiler is called an assembly and it contains a complete lossless binary encoding of the developer's high level source code. Because dotnet assemblies do not contain native machine code, they cannot be executed directly by the CPU. So in order to be executed, an execution engine of some sort must be present on the target machine which converts that IL into CPU understandable machine code on it just in time or jit basis before passing that code to the CPU for execution. The key part of this architecture is the just in time nature of the system, the machine code generation step is only done. The very first time a given method is invoked, the generated code is encased and used for all subsequent invocations. But the entire design of managed language systems is that the high level language, the intermediate language binary representation and the execution itself are all designed from the beginning to support the efficient generation of machine code if and only if that code ever needs to be executed. Hence the just in time terminology. So if a given method is never called no time or resources are spent generating the machine code equivalent of its intermediate language representation. But if a given method is called the translation of that method's IL in the machine code is triggered within the execution engine, which then removes itself from the call

chain for all subsequent invocations of that method. In other words, if that function is called 1000 times the translation of IL to machine code occurs once and then the other 999 invocations are carried out with no further involvement of the execution engine. And it is the jit compiler that also assists with runtime type safety. Since part of the ielt to machine code translation process involves verifying the type safety of the function that is being compiled and encoding other safety checks such as out of bounds array access attempts. This is what makes it possible for managed languages to exhibit the performance characteristics of compiled languages while still providing the convenience of automatic memory management safety and resilience to change that we looked at earlier.

The Common Language Runtime (CLR)

For .NET applications including, but not limited to those that are in C#, the execution engine is called the Common Language Runtime, or CLR. This is because IL is the common language that all .NET compilers emit, which means all managed languages such as C# and others share a common type system. There are actually several variations of the CLR spanning over two decades of .NET evolution and supporting various hardware platforms and device operating systems. Another course in the library provides a great overview of both the history and current set of options, so I'll reference that course at the end of this module. The two most common versions of the CLR today are the most recent version of what is simply referred to as .NET, which is the cross-platform version of the CLR we are using in this course, and the legacy version of what is referred to as the .NET Framework, which is spelled with a capital F and is specific to the Windows operating system and therefore not cross-platform. In general, all new application developments should target the cross-platform version of .NET in order to maximize your application's reach. Apps and libraries that target the cross-platform version of the .NET CLR will still run on Windows, which is what I've been using in this course, but they will also run on macOS and several flavors of Linux. You can view the current list of supported operating systems [here](#).

Just-in-Time (JIT) Compilation Revealed

To a skeptic like me, the resilient and safe, but with native performance claim comes across as the waviest of hand-waving. That's a bold claim. If you are also a skeptic, or if you just like seeing how

things are implemented, this next demo takes a look under the covers of the CLR using some specialized tools and shows you just how this JIT compilation magic is performed. But I should warn you, you will see things like disassembled machine code and references to registers and call stacks, which is somewhat of a departure from the run-of-the-mill big picture course at Pluralsight. That said, it's okay if you've never seen any of those things before. The point of this demonstration is not to acquire an understanding of those things, but to prove the claim that C# can be both safe and resilient to change as a result of being a managed language, and yet still exhibit the performance characteristics of native code. However, if you are content to take my word for it or don't care to know exactly how the CLR pulls off its JIT compilation magic, feel free to skip directly to the clip that follows this demo. You can rest assured that you will not miss any of the big picture regarding C#, only a bit of proof that one of the biggest claims made in this course is, in fact, true. For the adventurous among you, let's dig in. To show you Just-In-Time compilation in action, I've created a very simple C# application that adds two numbers together and displays the results to the console. However, to give us a chance to observe the before and after aspects of Just-In-Time compilation, I'm calling a separate method named Add to perform that arithmetic. Similarly, I've strategically placed calls to Console.ReadLine both before and after the call to that Add method in order to pause the application and give me a chance to attach a specialized debugger that we can use to observe Just-In-Time compilation in action. I'll build the application, but won't run it from here. Instead, I'll switch to the debugger where the remainder of this investigation takes place and position a console window where I'll be running the demo application alongside that debugger so that everything is visible at the same time. The debugger I'm using, which is also from Microsoft, is called windbg. This debugger is not included with Visual Studio, but you can download and install it for free at this address. To get started, I'll launch the demo application I just built in the console. Notice that our application, which is called jitviawindbg, has displayed its initial message to the console and is now blocked in a call to Console.ReadLine waiting for us to press Enter before calling the Add method. We'll use this moment to attach the WinDbg debugger to the application, which suspends the app being debugged in order to give you a chance to set breakpoints or perform other prep before resuming execution of the app. In our case, we'll use this moment to have a look around at the state of our application before JIT compilation is triggered for that method. I'm using WinDbg for this demonstration because it ships with a very special debugger extension that is developed by the same team that builds the CLR, which

means it knows the location and shape of the CLR's internal data structures. This debugger extension is called SOS, which stands for Son Of Strike. I'll spare you the history lesson on how it got its name, but you'll see me entering SOS to access its specialized commands, so I wanted to mention that. For example, if I enter `!sos.help`, the SOS extension will display all of the commands it makes available to us within the DBG environment. I'll use that syntax of an exclamation mark followed by `sos`, followed by a period, to access several of these commands. For example, executing `!sos.eeversion` will display version information about the .NET runtime or execution engine that this program is using. Recall that our application is currently paused on the first call to `Console.ReadLine` within our main entry point method, which occurs before the very first call to `add`. If what I said earlier is true, I should be able to find evidence that the `Add` method has not yet been Just-In-Time compiled. I can do that in two steps. First, I'll use the `name2ee` command to look up the `Add` function. The first parameter to this command is the name of a .NET assembly, and the second argument is the name of a type entity defined within that assembly. In our case, the assembly is named `gitviawindbg`, and the entity I want to look up is the `Add` method of the `Program` class. Note that this command confirms that our `Add` method has not yet been converted from IL in the machine code. Additionally, if I attempt to use the `sos` command to unassemble or inspect the machine code for the `Add` method, it will also tell me the `Add` method has not yet been Just-In-Time compiled, so it cannot convert the machine code for that method back into its more readable assembly language view. However, if the JIT compilation process is working the way I said it does, we should expect that the next time we use the same command, which will be after the call to `Add` has occurred, and theoretically triggered Just-In-Time compilation that we will see something different. Similarly, if I unassemble our `Main` method right now, we can see two things. First, `Main` itself has already been Just-In-Time compiled, which makes sense since we're currently paused within `Main` on its first call to `Console.ReadLine`. And second, we can see the call to our `Add` method from within `Main`. Don't worry if you're not familiar with reading a similar of language, which is what we're seeing here. `WinDbg` is kind enough to show us that this call instruction corresponds to a call to the `Add` method of the `Program` class. And these two instructions here represent the literal values 30 and 12 being passed as parameters to `add`. We can use the question mark operator in `WinDbg` to evaluate these hexadecimal expressions to confirm that. And if we use the `unassemble` command again to inspect the target of that call to the `Add` method, we find that we are actually calling a function named `PrecodeFixupThunk` within a library called `Core CLR`. This is the function that actually triggers

the Just-In-Time compilation process. Okay, we're almost there. Now that we've confirmed that JIT compilation of the Add method has not yet been triggered, I'll use sos to set a breakpoint at the start of the Add method. This is done using the bpmd command, which sets a breakpoint on a method based on its method descriptor address, which is the data structure I referenced in the unassemble command as well. This breakpoint will be triggered when our main entry point calls the Add method after the CLR has generated the machine code for our Add method, which will give us a chance to look around after the JIT compilation process has happened. Having done that, we're ready to let our application run full speed until that breakpoint has been hit. With the application running again, we can now press Enter in our console, which will result in Main proceeding with its call to Add, which should trigger the breakpoint we just set. Great. At this point, WinDbg has indicated that we've triggered that breakpoint, which means Just-In-Time compilation for the Add method has been triggered and completed. We can verify this by repeating our unassemble command from earlier to see the assembly language view of the machine code that was generated by the CLR. There's a lot going on here, but don't worry about the details. The key bit is that these three lines of code are the processor-specific instructions generated by the CLR's Just-In-Time compiler for adding the two parameters passed into the Add method to one another, and then returning the sum to the caller. And if we revisit the call to Add from within Main where we previously saw a call to a mysteriously named function called PrecodeFixupThunk, we now see something different. Instead of a call to PrecodeFixupThunk, which triggers Just-In-Time compilation, we now see that Main's call to Add goes to a single machine code instruction that itself jumps directly to the start of the Add method's native machine code implementation, which was generated by the JIT compiler. This confirms that the CLR has now removed itself from the call sequence between Main and Add, which means that all subsequent invocations of the Add method from anywhere in the application, not just Main, run with no CLR-induced overhead. It is this architectural design of having the C# compiler generate IL, which the CLR then compiles to machine code on a Just-In-Time basis that results in all of the type safety and resilience of managed execution while retaining the performance characteristics of natively compiled code.

The .NET Base Class Libraries

The last piece of the puzzle regarding managed execution and C# is the role of libraries, specifically the standard set of libraries that are included as part of a .NET installation. Whereas the CLR provides the runtime-critical services, such as JIT compilation and garbage collection that are essential to the proper execution of all .NET applications, other code that is more utilitarian and commonly, but not always, required exists in a suite of libraries that are part of the .NET installation on a given machine, and that application and library developers can take advantage of as needed. These libraries are called the Base Class Libraries, or BCL, and provide types for working with text, performing file.io, and numerous collection classes, such as lists, queues, and hash tables, and many other types. Together the CLR and the BCL provide the foundation for apps built using any .NET language, not just C#. Chances are, if you need a bit of nonbusiness or domain-specific utilitarian code, or even an entire application framework, it is provided for by one or more types in the BCL. The implications of what I just said highlight a bonus big picture aspect of what we've seen so far, which is that because all managed .NET code shares a common type system in the form of their intermediate language representations, any investment you make mastering the ins and outs of using the various Base Class Libraries, whether for building console apps, web apps, or mobile apps, is an investment in skill development that transcends C#, because a side effect of learning C# is that you learn the broader .NET platform. This also means that you will be able to choose the right language tool for the job. Gone are the days of making an all-or-nothing decision to choose the language that a large application or suite of applications will be built in. If C# is the best tool for a given job, or is the language that a given team is most proficient with, then that's the tool you can use. On the other hand, if VB or F# is the best tool for a given job or is the language a different team prefers, then the majority of your skills transfer because although there are syntactic differences from language to language that you would need to understand, any investment you've made becoming proficient with the various classes and application frameworks defined in the BCL is transferable.

Putting the CL in CLR

In the next demo, I'll demonstrate this kind of intra process common language capability in action by writing a C# application that leverages a mathematical library written in F# using the same BCL types to communicate. But as with the JIT compilation demo earlier, feel free to skip to the next clip if you are content to take my word for it. As with the previous demo, I've created a very simple C# application

that calls an Add method to compute the sum of two integer values and then display the result to the console. What's different is that I've implemented this Add method in a separate library named calc, which I've implemented using F# rather than C#. F# is a functional .NET language that is particularly well-suited for computationally-intensive problem domains, although my example here is admittedly rudimentary. Even if you've never seen F# before, it's possible to recognize what's going on here. The type statement here defines a class named Calculator, which is itself defined within a namespace called fscalc. The equal sign here indicates that this class comprises the following members, the first of which is a static member named Add, which takes two floating point parameters named a and b and then returns the result of evaluating the expression $a + b$. This means that the Add member is actually a method and its return value is the sum of a and b. The three additional methods named Subtract, Multiply, and Divide are then similarly defined. Building and running our program confirms that the interplay between the C# application and the F# library works as expected, and we can even use the Visual Studio debugger to single step directly from the C# application into the F# library and back again. This is made possible because all of the C# application and F# library are written in different high-level languages. Both share a common type system by means of the common intermediate language that both compilers generate and that the CLR knows how to work with. The fact that managed languages in .NET share a common type system in this way is what makes your investment in learning C# a transferable investment in the broader .NET platform, which allows you to choose the right tool for the job.

Conclusion

If you've made it this far, you've seen that the C# compiler, as well as all other managed .NET language compilers generates intermediate language, which is the common language of .NET and which the CLR converts into CPU-readable code on a Just-In-Time basis at runtime. And since the generated machine code is cached and reused for all subsequent invocations, it exhibits native performance characteristics subsequent to the one-time JIT compilation step. It's that JIT compilation process that enforces the runtime type safety of your code, ensuring the integrity of the application when it executes. And supporting your code are the Base Class Libraries, which provide a combination of basic types and sophisticated application frameworks. For more information on the difference between the .NET Framework, .NET Core, and the most recent cross-platform version of

the .NET runtime that was used in this course, as well as an introduction to some of the most commonly used libraries in .NET application development, be sure to check out this course on Class Libraries in C#. You can also learn a little bit more about garbage collection in .NET in this course, which I also referenced in the previous module. Now that we've taken a look at the key features of the C# language and its relationship to the common language runtime and Base Class Libraries, I want to give you a sense of what you can expect to see moving forward as the language continues to evolve and improve.

The Constant Evolution of C#

Introduction

Welcome to the final module of this course, the Constant Evolution of C#. As a developer, or aspiring developer, one of the realities you face is dealing with constant change. Hot texts appear almost daily. Some become widely adopted. All except the really bad ideas that are dead on arrival evolve over time, and then eventually they fade into obscurity to be replaced by the next new hotness. As a technology, C# is no different. It debuted in general release in 2001, has undergone constant change since then, and will undoubtedly undergo more change in the future. As this is a big picture course, I'm not going to provide an exhaustive history lesson on the evolution of C#, but what I do want to do is show you some select examples of change that C# has undergone over the years so that you can get a sense of what to expect moving forward should you decide to invest your time learning more about C#. Because while I don't know exactly how C# will look five years from now, I am confident that it will look different, yet familiar to how it looks today.

Principles of C# Evolution

Contrary to what every stockbroker will tell you, the past actually is a pretty good indicator of the future, at least as far as technology evolution goes. So before I round out this aspect of the C#: The Big Picture by showing you some specific examples of how C# has evolved, I wanted to share a few of the truisms that I have observed over the last 20+ years of C#'s evolution so that you can have a

sense for what to expect moving forward and hopefully take comfort in how C#'s future evolution is likely to play out. First, changes tend to be incremental or evolutionary, rather than disjointed. And as those new features become available, you can choose whether and when to adopt them in your code, which means that new language features tend to be non-breaking and not disruptive. Commonly, changes stem from the current idioms and best practices adopted by the broader community of C# developers, appearing as first class language features and future versions of the tools. And finally, if and when you do adopt new features, the result will generally be code that is simpler, faster, safer, or possibly all of those things. For these reasons, it may have been more appropriate for me to name this module The Conventional Evolution of C# because the organically formulated conventions of the broader C# community play an instrumental role in how the language of C# changes over time.

Top Level Programs

With those principles of C# evolution in mind, let's take a look at a few examples from C#'s history up to and including present day. We'll start with the canonical first application that developers commonly create when learning a new language. In this case, we essentially have a one-line C# application that displays "Hello, world!" to the console in a grammatically and historically correct manner, except that it's actually a seven-line program. This is because C# does not support the notion of global functions. Instead, functions always exist as methods on a type. So the simplest form of C# application requires a type, the Program class in this case, although you can name the enclosing type anything you like, in which to house a method, which, in the case of an application main entry point, is required to be a static method called Main. As long as the compiler only finds one method among all of an application source files matching the requisite shape for an entry point method, it will select that for use as the main entry point of the application. And although I could technically collapse this code to occupy only one line by removing the new lines, doing so would not make the point I'm trying to make. While one could argue that this particular language feature was long overdue, C# supports, starting with version 9, what are referred to as top-level statements, which can be used to implement a 1-line program with just 1 actual line of code. So as long as there is only one source file included in your project that leverages top-level statements, the compiler will happily produce the still required enclosing type and everything in your source file will simply be included within the Main entry point or the enclosing type that the compiler now generates for you. That said, the original style of writing Main entry point as part

of a top-level class, which can be called anything you like, is still supported. And if your program accepts command-line arguments, like this variation does, you can simply assume the existence of a variable named `args` and reference it like you normally would in order to access any command-line arguments passed to your program when it was started. The C# compiler simply declares it behind the scenes as if you'd done so manually. However, I've also slipped in one additional change, which is that I've now leveraged a feature of C# that was actually introduced a few years ago in version 6 called string interpolation. An interpolated string can be recognized by the dollar sign preceding a string literal and executable expressions embedded within curly braces inside that string. When interpolated strings are encountered, the C# compiler simply generates code that performs the operations within the embedded curly braces and then uses the resulting value or values of those statements to compose the string, which in this case is then passed to `Console.WriteLine`. While the string interpolation feature is not new, it's another example of features that can be opted into at your convenience.

Properties

Here's an example of an exceedingly simple two-dimensional point class. This class is similar to the point structure that was shown in an earlier demo in this course, except that we are using a class here with two private fields called `_x` and `_y` that hold the X and Y coordinates for the point, and two public properties called `capital X` and `capital Y` that allow callers to read and write those values. Now, this is a contrived example to keep things simple, but in practice, this is commonly done in order to do parameter validation on the value that is being passed into the property setter in order to maintain the internal integrity of an object. To keep the code on one screen, I'm not showing that type of validation here. The point is that this approach quickly became a recognized best practice for designing types, and so the C# language designers took steps to facilitate this type of design by implementing some syntactic enhancements. This is how that it became possible to implement that same pattern beginning with C# version 3. This version of the point class leverages auto-properties, which means we don't need to explicitly define private backing fields to hold the values we want to store. Instead, we can simply declare our publicly accessible properties and let the compiler handle that behind the scenes. Nor do we need to implement the canonical body of the getter and setter for each property. The compiler will implement those two in the manner that you would expect, although without

parameter validation. The result is definitely a lot cleaner than before, and you can always use the original syntax if this doesn't suit your specific needs. More recently, C# 9 codified support for a common pattern that involved the desire to define a property that has its initial values set by the caller when an instance of the type is initially constructed, but that could only be read by outside callers from that point on. A property that can be initialized once, but then only accessed in a read-only manner subsequently by callers outside of the type itself is implemented by using the `init` keyword instead of the `set` keyword to define the setter, as shown here. Init properties like this remain read-writeable from within the other members of the point type, just not from the outside. However, if a caller did not provide an initial value for an init-only property, the C# 9 compiler simply used a default value it deemed appropriate given the type of that property, in this case 0, which may or may not be the desired behavior. So beginning with C# 11, it became possible to signal to the compiler that callers are required to provide the initial value of a given property by adding a new `required` keyword to a property's declaration. As you'd expect, the compiler will then generate an error if a caller fails to provide an initial value for any such property.

Expression-bodied Members

Beginning with C# 6, it became possible to take advantage of a language feature called expression-bodied members to define the body of a type's members. In this case, this feature can be used to eliminate some of the structural syntax associated with implementing the `ToString` method. This is only possible here because the implementation of `ToString` consists of just a single expression. But whenever that's the case, which is fairly common, expression-bodied members allow the structural syntax related to defining a single expression member body to be omitted and replaced with the simple fat arrow syntax shown here.

Collection Expressions

Prior to C# 12, if you wanted to declare a collection and initialize it with some initial values, you were required to use the `new` keyword along with an array type expression, which is somewhat cumbersome. The only syntactic shortcut available at the time was to take advantage of compiler type inference, which allows you to omit the redundant array type portion of the expression. Meh.

Beginning with C# 12 and .NET 8, the compiler supports collection expressions, which allows you to use a more natural array syntax to express the initial set of values that should be used to initialize the collection on the left-hand side. Related, C# 12 also adds a spread operator that allows collection expressions to reference other collections as a shorthand for indicating that all of the elements of a referenced collection should be used to initialize the new collection. Here we're using the spread operator twice to indicate that the `fourStooges` variable should be initialized with the values from both the `twoStooges` and more `Stooges` variables.

Relational Patterns

Okay, last one, and for the record, forcing you to squint is part of the point I'm making here. What we have on the left is a `Person` class that defines a read-only property named `Generation`, which returns an enumerated type also called `Generation`. The `Generation` enum represents each of the generational cohort nicknames as they are currently defined in the US. Since generational cohorts are determined based on someone's birth year, not their age, the implementation of the `Generation` property performs a series of simple tests to identify the range within which someone's birth year would result in them being labeled with each of the defined nicknames. For example, individuals born from 1946 through 1964 are referred to as Baby Boomers. Individuals born from 1965 through 1980 are referred to as members of Generation X or Gen X, and so on, through the generation born from 2025 to 2039, which will be called Gen B or Gen Beta. As you can see, if you aren't watching this video on a mobile phone, implementing this type of relational pattern in this manner is fairly arduous and certainly unpleasant to read. Thankfully, this is how that same logic can be implemented as of C# 9. This version of the read-only generation property starts out by taking advantage of the expression-bodied member syntax we looked at a moment ago. The expression being used here is called a switch expression, which specifies a value to test followed by the `switch` keyword, which is a block construct enclosed in a pair of standard curly braces. Within the body of the switch expression, we can simply specify the relational tests that should be applied to that value being tested. Because the switch expression defines the value being tested, we don't need to repeatedly include the value to test like we did with the standard if-else construct. We are also using a new conjunctive and logical pattern to test whether multiple tests match. The value to return for a given pattern or combination of patterns is specified to the right of the fat arrow operator. Lastly, we specify the value to return when

none of the relational tests match by using something called the discard operator, which was introduced in C# 7. As with the other evolutionary improvements we've looked at, you aren't required to leverage this new syntax. However, these new features result in code that is much less laborious to write, and in more ways than one, much easier on the eyes, all with the same performance characteristics of the classic approach.

Module Summary

While there are many, many more examples of C#'s approach to feature evolution, I hope that the small sample we've explored here gives you a sense for what to expect should you choose to adopt C# as your next programming language. I think it's fair to say that you can expect to see continuous improvement of the language in a manner that doesn't disrupt your productivity and gives you control over the timing of new feature adoption in your code base, resulting in code that is increasingly simple, safe, and, in some cases, faster. Throughout this module, I either mentioned or alluded to these courses in the Pluralsight library that you can check out for more information on several of the topics I just touched on.

Community Resources

Having only just scratched the surface of C# in this big picture course, I want to mention a few additional resources that will prove helpful for those of you who wish to further your study of C# and the .NET platform. Docs.microsoft.com is the jumping off point for all of Microsoft's reference documentation, and you'll find everything related to the overall .NET platform here, and the documentation specific to the C# language here. Microsoft also hosts a great developer blog at devblogs.microsoft.com with technical content from Microsoft insiders, as well as other experts from around the industry. The csharp category tag shown here will take you directly to posts that are specific to the C# language. Like many other open source technologies, you can also find an enormous amount of information readily available on GitHub. In particular, the dotnet/core repository is where you'll find all of the technical details about the .NET runtime, including stable and preview releases, its road map and links to related community resources. And the dotnet/csharp repository is where you'll find the ultimate source of truth on the C# language features, including detailed

language specifications, historical information, and future feature proposals under consideration. The `dotnet/csharp` repo also includes a discussion forum where the community can interact with one another and the C# language design team about C# language features. For numerous additional community resources, be sure to check out the platform community page at dotnet.microsoft.com, which serves as the jumping off point to a whole host of additional resources related to .NET, including, but not limited to, C#.

Conclusion

This brings us to the end of this big picture tour of C#'s key features. I hope that you found this course informative, revealing, and, more importantly, helpful for gaining a high-level understanding of the core characteristics of C# and its runtime environment, and perhaps prepared to continue on a learning journey towards personal proficiency with C# armed with a sense for what to expect as C# continues to evolve. I wish you all the best wherever your journey leads.